STUDENT LEARNING NOTES

Premium Talent Accelerator Program — Student Learning Notes

Developed by Talenciaglobal | **Facilitator:** The Mentor | **Training Date:** 16 October 2026 | **Source:** Three recorded training-session transcripts (consolidated) | **Next Working Session:** Wednesday, 9:30 a.m.

About These Notes

This document consolidates the key concepts, examples, student contributions, working principles and action items from the training. It is a structured practical reference for students working on complex, AI-enabled enterprise applications — not a verbatim transcript.

How to use it: Read the document as a connected method rather than a list of independent tips. The central message is that AI has made code generation much easier, so engineering value increasingly comes from defining the problem precisely, designing the right data structures and schemas, validating decisions, building quality into the process, and delivering reliably.

Use these principles when starting a new feature, module, project or hackathon assignment. Apply the schema-first and data-driven approach before asking an LLM to generate large volumes of requirements, tests or code.

Note on illustrative material: Examples and numbers from the session (e.g., table/row counts, token budgets, test counts, iteration counts) are illustrative context unless explicitly presented as a target for an exercise. They are not universal engineering benchmarks.

Note on terminology: The transcript's audio rendering varies between "QFC" and "QFE." The confirmed term is **QFE — Quality First Engineering**. All references in these notes use QFE.

Why This Matters: The Industry Reality in Numbers

The Mentor's guidance is not abstract theory. It is a response to a measurable crisis in software delivery. The following figures — drawn from independent industry research — explain *why* the methods in this document matter.

$2.41T

Annual cost of poor software quality

to the U.S. economy, according to CISQ

$1.81T

Operational software failures

crashes, outages and logic errors — the single largest contributor

64%

Rise in cybercrime losses

enabled by software vulnerabilities between 2020 and 2021

$260B

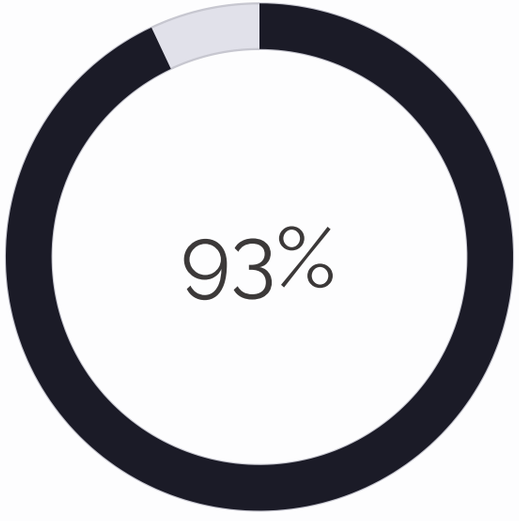
Unsuccessful development projects

estimated cost in CISQ's 2020 report alone

The implication for this program: Quality First Engineering (QFE) is not a discipline for its own sake. The gap between "it runs" and "it is correct, maintainable and trustworthy" is where trillions of dollars of value are lost or protected.

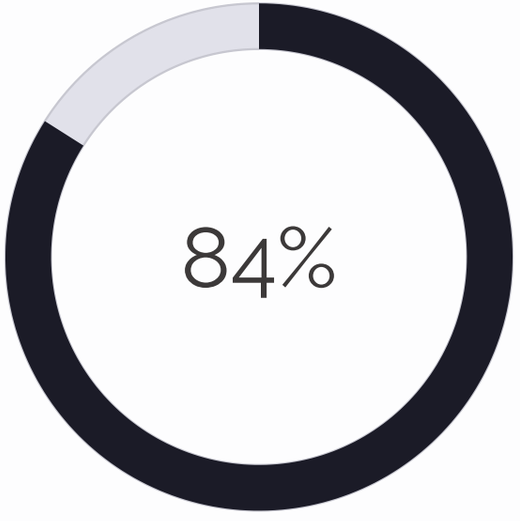
Engineers Spend Only 16% of Their Time Actually Coding

This is perhaps the most striking statistic for students entering the profession:



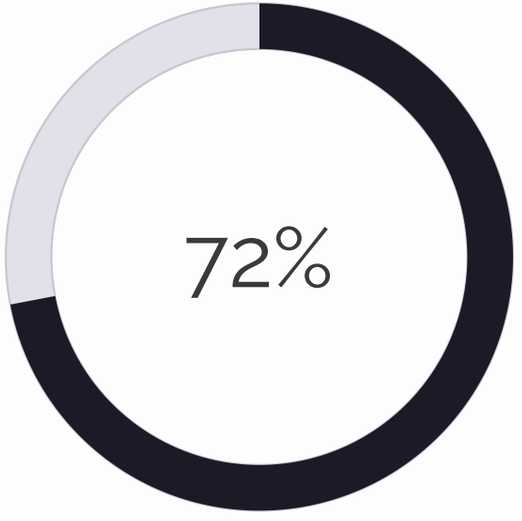
Say coding is most rewarding

yet they spend just 16% of their time writing code and building new features



Time consumed by other work

maintenance, patching, technical debt, vulnerability management and administrative tasks



Competing demands

of engineers say competing demands prevent them from prioritising new feature work

Two-thirds of engineers report encountering technical debt "all the time."

The implication for this program: The schema-first, quality-first approach is designed to reduce the rework and technical debt that consumes 84% of engineering time. If you learn to define structures properly *before* generating code, you spend more of your career on the 16% that creates new value.

Project Delivery Is Failing at Scale — and Has Been for Three Decades

The Standish Group's CHAOS data tells an uncomfortable story that has not meaningfully improved since 1994:

~35% of projects succeed

meet time, budget and feature goals; roughly 45% are challenged; close to 20% fail outright

16.2% success in 1994

The original 1994 CHAOS Report; 2020 data implied roughly 16% — remarkably close despite decades of new methodologies, tools and frameworks

90% vs <10%

Small projects achieve ~90% success; large projects succeed less than 10% of the time

13% government success

Only 13% of major U.S. state and local IT projects succeed; only 13% of large federal IT procurements over $6M succeed

Why projects actually fail: The top reasons cited across CHAOS Report years have remained strikingly consistent — **lack of user input, incomplete requirements and specifications, and changing requirements**. These are *not* technical failures or talent gaps. They are failures of **discovery, definition and discipline** — precisely the skills this program develops.

The implication for this program: The Mentor's emphasis on "discover before defining," schema-first design, and stakeholder/persona schemas directly attacks the root causes of three decades of project failure.

Software Outages Are Endemic — and Expensive

Two-thirds of organisations expect a major software disruption in the coming year, with nearly **one-quarter characterising their risk level as "extreme."**

For **two in five organisations**, annual costs from software outages exceed **$1 million**. In **financial services**, nearly **half report losses exceeding $5 million per year**.

The **CrowdStrike outage** of 2024 — caused by a single faulty security update — produced estimated direct losses of **$5 billion** among Fortune 500 companies, cancelled thousands of flights and stranded passengers worldwide. Cybersecurity consultant Troy Hunt called it *"the largest IT outage in history"* and added: *"This is basically what we were all worried about with Y2K, except it's actually happened this time."*

$5B

CrowdStrike 2024 losses

Direct losses among Fortune 500 companies from a single faulty update

$5M+

Financial services outage cost

Nearly half of financial services firms report losses exceeding this annually

The implication for this program: The test suites, release gates and quality scorecards the Mentor described are not bureaucratic overhead. They are the mechanisms that prevent the next CrowdStrike-scale failure.

Data Is Exploding — But Most of It Drives No Action

240ZB

Data created in 2026

The world created roughly 230–240 zettabytes of data in 2026

28%

Enterprise data growth

annually — cloud data growing 36% and edge data 33% per year

11K

Daily security alerts

average security operations center handles ~11,000 alerts a day

19%

Worth investigating

Only about 19% of those 11,000 daily alerts are ever worth investigating

Most organisations now store more than **5 petabytes** of unstructured data; **40% store more than 10 petabytes**.

The implication for this program: The Mentor's data-first principle — decide what information the product must represent, what insights it should produce, and what evidence demonstrates it works — is exactly what separates a data-rich application from a *useful* one. Volume without structure is noise.

Data Scientists Spend More Time Cleaning Data Than Analysing It

The "80% of time cleaning data" claim has circulated for over a decade. The reality is more nuanced — and more useful:



Anaconda survey

Data scientists spend roughly 45% of time on data preparation, with cleaning and organising alone accounting for over 26% of the average workday



CrowdFlower (earlier surveys)

60% on cleaning and organising, reaching 80% when data collection and labelling were included



Kaggle

Around 15% specifically on cleaning, using a narrower definition

The honest range is 25–80%, depending on what you count — but even the conservative end is expensive. For a team of 10 data scientists at an average $135,000 salary, **45% time on preparation equals $607,500 per year**; at 80%, **$1,080,000 per year**.

The implication for this program: The Mentor's insistence on defining **schemas and data dictionaries first** is a direct response to this waste. If the structure is right, preparation time collapses. If it is wrong, you pay for it repeatedly.

Enterprises Run More Applications Than They Can Govern

125+

Average SaaS apps per org

By 2024, the average organisation used over 125 SaaS applications

650+

Tools in large enterprises

Larger enterprises ran 650+ tools, adding six new ones monthly

46% of apps in the average company's portfolio have **low or poor risk scores**.

The implication for this program: With this level of sprawl, the engineer's value is not in adding another application. It is in **defining the data and schemas that connect systems coherently** — and in building the validation that prevents risk from compounding.

The Cost of a Defect Multiplies by 100× in Production

1× — Design/Development

Catching a defect at the design stage costs the baseline amount

10× — Testing

A defect is 10× more expensive to fix in testing than at design

100× — Production

A defect is 100× more expensive to fix once it reaches production

Defect density benchmarks illustrate the quality gap: **critical software** targets **less than 0.1 defects per KLOC**; **high-quality enterprise systems** around **1–3 per KLOC**; **typical business applications** up to **10 per KLOC**. A meta-study of 109 projects (2002–2010) found a **mean defect density of 7.5 per KLOC** and a **median of 4 per KLOC**.

The implication for this program: The Mentor's test-case graph, release gates and quality scorecards are not optional extras. They are the mechanisms that shift defect discovery leftward — from production, where fixes cost 100×, to design and testing, where they cost 1×.

The AI Paradox: More Code Generated, More Quality Risk

AI tools have made code generation dramatically easier. But the industry data above explains why this is **not** purely good news:

84% time on maintenance

When engineers already spend 84% of time on maintenance and debt, generating *more* code without adequate structure and testing compounds the problem.

Two-thirds face outage risk

When two-thirds of organisations already face outage risk, accelerating code production without QFE discipline increases exposure.

Only 35% of projects succeed

When only 35% of projects succeed, generating volume faster does not fix the underlying discovery and definition failures.

The implication for this program: The Mentor's central argument — that the higher-value engineering work is **discovering the right structure, defining it clearly, and validating the output** — is the only sustainable response to the AI paradox. Code generation is commoditised. **Schema design, data understanding and quality assurance are not.**

Executive Summary

1 AI-assisted development has reduced the effort required to generate code.

The differentiator is no longer simply how quickly a developer can write code or how many frameworks they know.

2 Strong engineering begins with functional and non-functional clarity.

Teams must understand what the system must do, who uses it, what constraints it faces, and what quality, security, scale and reliability it must achieve.

3 System design and architecture still matter.

Technology selection may be easier to change or learn with AI support, but foundational architectural choices should be deliberate and justified.

4 Estimation, prioritisation, time discipline and accountability are core professional capabilities.

A team must understand the size of the work, its own capacity, the deadline and the cost of failure.

5 Data must be a first-class design concern.

Data should shape how the application is understood, designed, demonstrated and tested — not be loaded in near the end.

6 Compelling demonstrations tell a story through realistic, high-value data.

Attractive screens and many features are not enough if the data does not show real utility or business impact.

7 A major part of human contribution in AI-era engineering is defining schemas.

Deciding how information should be organised, classified, related and validated before asking AI to generate outputs.

8 The recommended working sequence

Discover before defining → define nonlinear schemas → challenge with multiple sources → validate outputs against data and test cases → iterate → make the process repeatable.

9 Quality must be systemic, not dependent on the individual developer.

Strong test suites, data dictionaries, explicit rules and repeatable checks prevent personal assumptions or bias from becoming product defects.

1. The Changing Reality of Software Development

1.1 AI changes the economics of building software

The Mentor opened by asking students how they would approach building an end-to-end enterprise application when AI tools can generate substantial applications from detailed product requirements. He referenced the example of reverse-engineering a known application such as Spotify into a detailed **Product Requirements Document (PRD)**, then using that PRD to generate a new implementation with very little manual coding.

The point was **not** that complex software has become effortless or that engineering knowledge is obsolete. Rather, **detailed context makes AI generation dramatically more effective**. If product behaviours and constraints are specified with great precision, code generation becomes much easier. Real-world requirements, however, are rarely that complete.

1.2 What students identified correctly

- **Amogh** highlighted the need to identify **functional and non-functional requirements** in detail *before* making system design decisions.
- He noted that expected system load (e.g., number of profiles, applicants, applications) should inform non-functional requirements and architecture.
- He cautioned against **blindly accepting an LLM's recommended system design**. Architectural decisions should be researched, reasoned about and verified by the team.
- In a big-data/IoT example, he described the importance of foundational **ingestion and streaming** choices — e.g., an **MQTT broker** for device messaging and **Apache Kafka** for durable, distributed event streaming. These are choices to understand early rather than casually change later.

1.3 Technology selection is important — but no longer the same bottleneck

A few years earlier, teams spent considerable time debating programming languages, frameworks, databases, cloud platforms, licensing, test automation tools and hiring for specific stacks. A team might have chosen Java/Spring Boot, .NET, React, Angular, Python/FastAPI, or a particular database partly because the available developers already knew it.

AI tools make it much easier for capable engineers to learn and work with unfamiliar technologies. This **reduces** the constraint imposed by each individual's existing stack experience — it does **not** make technology choices irrelevant. The selected stack must still fit the problem, architecture, performance, cost, security and operational requirements.

Practical takeaway: Do not start with *"Which language do I know?"* Start with *"What problem are we solving, what constraints must we satisfy, and what design best fits those constraints?"* Then choose and validate the technology.

2. Start with Clarity: Requirements, System Design and Architecture

2.1 Functional clarity

Functional clarity describes **what the system must do**: the users/actors, the actions they perform, the triggers that start work, the business rules that govern it, and the expected outcomes. Requirements should be specific enough that a team can determine whether the behaviour has been implemented correctly.

2.2 Non-functional clarity

Non-functional requirements describe the **conditions and qualities** under which the system must operate: scale, concurrency, performance, latency, security, privacy, accessibility, usability, reliability, recovery, maintainability and deployment considerations. Identify these early — they influence architecture and testing.

2.3 System design and architecture

System design translates requirements and constraints into a workable technical solution. Architecture covers decisions about component boundaries, data architecture, application structure, UI, security, testing, deployment, and whether a monolithic or service-oriented approach is appropriate. Students also raised CI/CD and DevOps, hardware constraints, documentation, testing and scalability.

The Mentor's message: **think through the whole system**, not only the code that is easiest to see. Foundational decisions affect later work, and changing them late can be expensive.

2.4 A practical pre-build checklist

1. What problem and business outcome are we addressing?
2. Who are the users, actors and stakeholders?
3. What are the functional requirements, business rules and expected outcomes?
4. What are the non-functional requirements, and how will they be measured?
5. What scale, concurrency, data volume and usage patterns should be expected?
6. What are the major data domains, entities, relationships and lifecycle rules?
7. What architectural boundaries and integrations are required?
8. What security, privacy, access-control and compliance needs apply?
9. How will we test functionality, quality attributes and failure scenarios?
10. How will the application be deployed, monitored, recovered and maintained?
11. What are the estimates, milestones, dependencies, risks and acceptance criteria?

3. Estimation, Prioritisation and Delivery Discipline

3.1 Estimation is a core engineering skill

Estimation is one of the most consequential parts of delivering software. A feature list or a story-point total does **not**, by itself, tell a team whether it can deliver by a particular date. The team must understand the **size and complexity** of the work and relate it to its **actual capacity**.

The Mentor discussed **story points**, **T-shirt sizing** and **time estimates**. For example, a backlog of 100 features and 350 story points suggests a substantial body of work, but delivery time depends on team capability, tools, familiarity, workflow and quality expectations. **AI may reduce coding time sharply, while requirement understanding, validation, integration and testing still require deliberate effort.**

Terminology note: Story points are *relative* units of effort/complexity, not hours. Velocity (story points completed per sprint) is team-specific and only meaningful when compared against that same team's history. Do not treat story-point totals as time estimates.

3.2 Under-promise and over-deliver — without hiding uncertainty

A student suggested estimating with a conservative upper bound so the team could finish early and delight the client. The Mentor agreed with the principle but added the competitive reality: teams operate under deadlines and commercial constraints, and another team may offer faster delivery. The goal is **not** to pad estimates arbitrarily — it is to make **realistic, evidence-based commitments** and manage risk.

3.3 Deadlines change the way a team must work

A two-month delivery window leaves less room to recover from missed milestones than a six-month window. Illness, holidays, dependencies, integration issues and unexpected defects do not remove the commitment. In customer-facing engineering roles, missing a promised date can trigger escalation even without a formal legal penalty.

- Treat deadlines and agreed acceptance criteria seriously.
- Break work into smaller, reviewable deliverables and identify dependencies early.
- Do not assume defects can always be fixed later — ask *when* fixes can realistically be completed.
- Use system-level thinking to reduce late trial-and-error.
- **Communicate risks early** instead of surprising the team or customer at the deadline.

3.4 Prioritise by business value and urgency

Spending time enthusiastically on a low-value feature while a critical customer need remains unmet is a failure of prioritisation. Priority can come from business value, but **urgency also matters**. A feature may not be the largest revenue opportunity and still be essential because users need it tomorrow — for example, a form required to submit a loan application.

3.5 Accountability

Accountability means owning a commitment, clarifying what must be delivered and by when, confirming the expected result, and following through. The Mentor framed it as a personal discipline: ask **"By when do you need this?"** and **"Is this the outcome you expect?"** before making a promise. Dedication and hard work become meaningful when connected to ownership of an outcome.

4. The Central Theme: Data Is Not an Afterthought

4.1 The data-first idea

Applications fundamentally **store, retrieve, transform and apply rules to data**. Teams often focus heavily on requirements documents, UI mock-ups, architecture and code, then load data near the end to make the application appear functional — making data feel like a final implementation detail even when the product's real value depends on it.

The intended shift: make data a **first-class design concern from the beginning**. Decide what information the product must represent, what high-value scenarios it must support, what insights it should produce, and what evidence would demonstrate that it works.

4.2 Why a polished demo may still fail to impress

Enterprise applications with attractive screens, many features and elaborate workflows can still fail to create a compelling buying moment. In demos, presenters often enter placeholder values such as "AAA" or "123" because realistic data is tedious to enter. The demonstration then shows that buttons and forms work — but not the application's real utility.

A more effective demonstration uses **carefully selected, realistic data that reveals a meaningful story**. In the healthcare example, the demo could show a diabetic patient with multiple complications — including complications that might otherwise go undetected — and explain how the information supports a meaningful decision. **The value comes from what the data reveals, not merely from the screen layout.**

4.3 Data can be the product's defensible value

The Mentor used a hypothetical product called **Talentra**, imagined as a successful talent platform competing with established platforms several years in the future. He pictured not just many UI screens or microservices, but a rich **information asset**: many users, profiles, jobs, relationships, analysis and intelligence. He gave illustrative scale figures — roughly **1,350 tables and 155 million rows** — to make the imagined data footprint tangible.

These figures are not a required design target. The point is that the platform's value depends on the information it holds, the intelligence it derives, how safely it manages that information, and whether it can prove its impact. If a product claims to improve people's careers, it should show evidence or scorecards demonstrating before-and-after outcomes.

4.4 Data integrity is a business-critical responsibility

A service failure may be recoverable; **corrupted, lost or incorrect data can undermine decisions across the business**. If organisations rely on an application's data, incorrect information can cause repeated bad decisions at scale. (This section of the source transcript was truncated mid-sentence; the principle above captures the conveyed intent.)

4.5 Synthetic data is useful — but it is not the whole problem

Modern synthetic-data tools can generate large volumes of records far more easily than in the past, reducing the need to spend months merely collecting test records. But **volume alone is not enough**: data must be realistic, structured, relevant to the domain, internally consistent and useful for testing the scenarios that matter.

The challenge is not just generating rows. It is deciding **what the rows should represent**, what relationships and edge cases matter, what quality standards apply, and whether the resulting data supports credible analysis.

Data-first principle: Design the important data and scenarios early. Use them to inform the schema, the requirements, the demo, the analytics and the test suite — not simply to populate the application after it has been built.

5. A Practical Example: Build the Insight Before the Interface

The Mentor described a **university curriculum-analysis portal**. Its source was an Excel file containing curriculum information; the team did **not** begin by building a large set of data-entry forms. Instead, it focused on assembling information about universities and courses and presenting analysis useful to the user.

The example included analysis of:

- Course counts, credits and teaching hours
- Semester distribution
- Theory versus practical work
- Project and internship components
- Electives
- Foundational versus advanced courses
- Credited versus non-credited courses
- Subject-category distribution

Filters and comparative views could then support further analysis.

What this example demonstrates

Assemble and curate first

The most valuable first step may be assembling and curating the right information, not building CRUD screens.

Existing data is a starting point

An existing spreadsheet or dataset can be a productive starting point if it already contains the core information needed for the intended analysis.

Insights without new entry

A useful application can expose patterns and insights from well-organised data without requiring users to enter every record through a new interface.

Incremental features

Once the data and analytical structure are clear, additional features and interfaces can be added incrementally.

Quality of source data matters

The quality of the output depends on the quality, completeness and structure of the source data and the analysis applied to it.

6. Schema Design: Where Human Thinking Becomes More Valuable

6.1 The new engineering task is not merely prompting

The Mentor showed a **requirements-analysis docket** generated from a source requirements document. It contained multiple sections — scope, functional and non-functional requirements, data domains, traceability, readiness, execution planning, decisions and issues. The functional-requirements section alone contained fields such as ID, domain, module, detailed description, actor, trigger, business rule, expected outcome, priority, coverage, tier and obligation.

The lesson: generated documents may contain many fields whose meanings are not obvious from their names alone. Engineers must inspect actual examples, understand what each field means, determine which values are valid, and decide whether the structure is useful.

Passing a document into an LLM and accepting the output without understanding it is like acting as a **courier who never opens the package**.

6.2 What is a schema?

In this training, a **schema** is the deliberate structure used to organise information for a particular activity. It defines the major categories, subcategories, fields, relationships, classifications and constraints that make the information useful. A schema is **more than a list of column names** — it is a model of how the team wants to understand and work with the domain.

The Mentor recommended **nonlinear schemas**, often visualised as **graphs or trees**, rather than forcing everything into a very wide linear spreadsheet at the beginning. A graph can represent parent categories, subcategories, attributes and relationships. The structure can later be **mapped** into relational tables or another implementation form when appropriate.

Technical clarification: A graph/tree is a *conceptual modelling* device. Relational tables, document stores or code structures are *implementation* forms. Mapping a conceptual model into an implementation is a deliberate design step (analogous to logical vs. physical data modelling) — it is not automatic, and normalisation/denormalisation trade-offs still apply.

6.3 Why schema design requires judgement

A useful schema requires:

- **Context:** understanding the business, the task and the intended output.
- **Classification:** grouping related concepts sensibly; separating genuinely different concepts into distinct branches.
- **The right level of detail:** splitting a category into subtypes only when the distinction affects behaviour, validation or reporting.
- **Decisions** about mandatory vs. optional fields, source-derived vs. explicit information, valid values and status.
- **Testing against examples:** a structure that looks elegant in isolation may fail when real requirements or edge cases are applied.

6.4 The graph example from a student

Lakshya shared a graph organising test cases into categories and subcategories, with child nodes describing what each test case covered. The Mentor encouraged presenting the **upper branches and subtrees first**, rather than every leaf item at once — making it easier to evaluate the organising logic before reviewing details.

General review technique: first inspect the top-level categories and the first few levels of the schema; then drill into individual fields and test cases. Reviewers should be able to explain **why** the structure exists and **what** each branch covers.

Key distinction: AI can help generate a requirements docket, test cases or code. The engineer's higher-value responsibility is to decide **what a correct, useful and testable output should look like** — and to verify that the generated result conforms to that design.

7. The Working Method Taught in the Sessions

The third transcript presents a compact set of working rules. The reliable steps are summarised below.

01

Define nonlinear schemas

For each major activity, first define the structure that will organise its output: requirements analysis, stakeholders and personas, goals, feature discovery, test cases. Use a graph or tree to represent major categories, subcategories and their relationships. **Do not begin by asking an LLM to generate a large output with no agreed structure.**

02

Discover before defining

Do not jump immediately to a final schema based only on first impressions. Explore source documents, inspect examples, identify domain concepts and use available expertise. Ask questions and discover what the structure needs to represent **before** finalising it.

03

Use multiple sources of truth

Check a proposed approach with more than one source — including multiple LLMs such as ChatGPT, Claude and DeepSeek — to challenge assumptions and make progress. **This is not permission to accept whichever answer appears most confident.** Compare suggestions, resolve disagreements and defend the final design.

04

Validate the schema and its generated output

Once the schema is defined, provide it as an **explicit constraint** when asking an LLM to generate the relevant artefact. Then inspect the output against the schema: validate field meanings, required values, coverage, relationships, source traceability and consistency. Use test cases and representative data to expose gaps.

05

Improve through iterations

Review and refine schemas multiple times — often **three iterations** as a practical recommendation, not an inflexible rule. The first draft is unlikely to be perfect. Each cycle should expose omissions, confusing categories, weak definitions or inadequate coverage. Avoid endless polishing if the result is demonstrably fit for purpose; do not rush past meaningful review.

06

Build a repeatable process

Apply the improved method to other modules and projects. Share schemas across teams, review each other's work, and incorporate useful ideas. Repetition and feedback turn a personal approach into a common method others can use consistently.

07

Optimise for quality, time and cost

The long-term goal is not to produce an artefact once, but to produce **reliable outputs repeatedly**, with less rework and more efficient use of AI tools and tokens. The Mentor gave an illustrative ambition of building a large application while keeping AI usage within a stated cost budget (see §16.9). **Treat that figure as a training example, not a universal benchmark.**

8. Process Maturity: From Ad Hoc to Optimised

The Mentor described a maturity progression to help teams assess how they work:

Stage	What it means	Evidence to look for
1. Undefined	Work is largely improvised; individuals choose their own approach and discover the process while executing.	Inconsistent deliverables; repeated confusion; success depends heavily on who does the work.
2. Defined	The team has documented phases, steps, schemas, responsibilities and expected outputs.	People can explain the process and follow a shared structure.
3. Repeatable	The defined process has been used successfully more than once and can be applied to another module or project.	Comparable outputs across teams; reusable templates and checks; fewer avoidable omissions.
4. Mature	The process is well understood, disciplined and integrated into how the team works.	Strong reviews, reliable hand-offs, clear ownership and predictable execution.
5. Optimised	The team improves the process for quality, speed, cost and reduced rework.	Measured improvements; automation where appropriate; efficient use of tools; reliable outcomes at scale.

Warning: avoid living permanently in firefighting mode. A first definition will not be perfect; implementation feedback should improve it. Repeated cycles — potentially around **six applications** of the define/implement/review loop — can reveal what works and make the process repeatable. The exact number is illustrative, not a fixed rule.

9. QFE (Quality First Engineering) and Test Design

9.1 Quality must be designed into the process

Quality should not be left to the end of the project or treated as an individual developer's personal virtue. **The system should make poor-quality releases difficult** by requiring structured checks, data validation and adequate test coverage.

9.2 Test cases are a safety net

Build a **structured test-case graph**. Organise test cases around meaningful categories and map them to the requirements or behaviours they cover. The goal is **not** a large count of unstructured tests — it is to understand **coverage** and ensure tests protect the important behaviour of the system.

- Map tests to requirements, business rules and expected outcomes.
- Include normal scenarios, boundary conditions, invalid inputs, failure paths, and relevant security/privacy conditions.
- **Test the data itself:** field formats, required values, consistency, relationships and edge cases.
- Maintain a **data dictionary** so fields, meanings, units, valid values and constraints are explicit.
- Use the test suite as a **release gate**, rather than relying on confidence or a successful demo alone.

9.3 Make quality independent of the individual

On dataset bias: people may unintentionally introduce assumptions or bias. The answer is **not** to assume every individual can be made perfectly objective before software is built. Instead, **establish a system of checks** that detects errors and constrains poor outcomes regardless of who writes the code.

He used a large test-suite example: a product should not be released unless its required tests pass. **The exact test count mentioned is an illustration of a strong quality gate, not a prescribed number for every project.**

9.4 Maintainability matters alongside time and space complexity

The Mentor expanded the familiar **time- and space-complexity** discussion by adding **maintainability**. AI can help refactor code and improve structure, but engineers remain responsible for code that is readable, well organised, consistent with best practices and maintainable by the next person. A solution that runs quickly but is difficult to understand or safely change is not complete engineering.

QFE mindset: Do not ask only, *"Does it work on my machine?"* Ask, *"Can we demonstrate that it works correctly across the scenarios that matter, and can we maintain it safely?"*

10. Responsible Use of AI and Engineering Judgement

10.1 AI is a capability multiplier, not a substitute for understanding

Use LLMs and modern development tools — but generated output should be **understood, checked and improved**. Merely prompting an LLM and forwarding its result is not engineering.

10.2 Prompt quality and schema quality work together

Clear communication matters because AI outputs depend on the context and instructions supplied. But the session went beyond prompt writing: the team must also **specify the schema or structure** the output should follow. A well-defined schema makes expectations more explicit and validation easier.

10.3 Data bias and systemic safeguards

Assumptions can enter data and models unintentionally. The practical response: document the data; understand its origin and limitations. Test for problematic outcomes where relevant. Create **systematic safeguards**. Use multiple LLMs to challenge an idea — but recognise they do **not** guarantee truth or eliminate bias.

10.4 Domain knowledge remains essential

Understanding a complex domain such as healthcare can take substantial time. Domain terms, regulations, business practices and real user needs **cannot** be replaced by a generic technical solution. The team must acquire enough domain understanding to interpret data and requirements correctly.

11. Collaboration and Peer Review

The training was framed as a working discussion rather than a one-way lecture. Students were encouraged to speak, type notes into the chat, challenge ideas and share practical examples.

- Create schemas **collaboratively**, but make the logic visible so teammates can critique it.
- Ask one team to review another team's work. Reviewers learn both what to do and what to avoid.
- Compare schemas and reuse useful ideas rather than assuming the first version is final.
- Show the **top-level structure first**, then review details and leaf nodes.
- Ensure **every team member** understands the artefact, not just the person who generated it.

Collaboration exposes teams to multiple approaches and provides a practical way to improve consistency across modules.

12. Suggested Student Workflow for the Accelerator Project

This workflow translates the Mentor's guidance into a practical sequence. It is a **synthesis** of the sessions, not a verbatim checklist.

01

Understand the outcome

Write a concise statement of the problem, intended users and business outcome. Clarify what success looks like.

02

Study the source material

Read requirements, examples and existing artefacts. Record assumptions, unknowns, conflicts and questions.

03

Discover the domain and data

Identify real-world entities, relationships, key scenarios, data sources, quality risks and important edge cases.

04

Define nonlinear schemas

Create graph/tree structures for requirements analysis, stakeholder/persona information, goals/features and test cases as applicable.

05

Challenge the design

Ask domain experts and use more than one LLM/source to find omissions or alternative classifications. Resolve conflicts deliberately.

06

Define acceptance and quality criteria

Specify required fields, expected outcomes, traceability, non-functional requirements, data rules and test coverage.

07

Generate structured artefacts with AI

Provide the source context and the agreed schema. Request outputs that follow the schema instead of accepting unconstrained prose.

08

Inspect and validate

Check generated content against source material, schemas, real data examples and requirements. A polished document is not necessarily correct.

09

Iterate

Refine the schema and artefacts based on review and test results. Use multiple cycles where they add value.

10

Implement and integrate

Build the application using the selected architecture and technology, with security, deployment and maintainability in view.

11

Test and demonstrate

Run structured tests, validate the data, and demonstrate a meaningful scenario using realistic, high-value data.

12

Review delivery and learn

Compare actual effort with estimates, document defects and decisions, and update the process for the next module.

13. Student Action Items and Expected Working Habits

Immediate actions (discussed or implied in the training)

- **Create the five key nonlinear schemas** for your team's work: requirements analysis, stakeholders/personas, goals, feature discovery and test cases.
- **Review each schema more than once** — approximately three iterations as a practical target where helpful.
- **Inspect generated artefacts at the field level.** Understand what each field means and how its values should be interpreted.
- **Use peer review across teams** and exchange notes so the group converges on better, reusable structures.
- **Develop a structured test suite and data dictionary.** Test the data as well as the application logic.
- **Return with tangible progress** at the next working session. The Mentor explicitly asked students to use the time between sessions to create and refine the schemas.

Habits to cultivate

- Precision in thinking and communication.
- Curiosity: discover before deciding.
- Evidence-based reasoning rather than blind acceptance of AI output.
- Self-accountability and dependable commitments.
- QFE execution and disciplined validation.
- Willingness to review, revise and learn from other teams.
- Focus on meaningful business outcomes rather than activity, code volume or visual polish alone.

14. Important Terms Used in the Training

Term	Meaning in these notes
PRD (Product Requirements Document)	A structured description of what a product should do and the behaviours it must support.
Functional requirements	Statements describing system behaviour, user actions, rules and expected outcomes.
Non-functional requirements	Constraints and quality attributes such as performance, scale, security, usability, reliability and maintainability.
System design	The process of translating requirements and constraints into a technical solution.
Architecture	The major structure of the system, including components, boundaries, integrations and cross-cutting concerns.
Schema	A defined structure for organising and describing information, including categories, fields, relationships and constraints.
Nonlinear schema	A graph- or tree-like organisation representing nested categories and relationships without forcing everything into one flat list.
Data dictionary	A reference defining data fields, meanings, formats, valid values and constraints.
Synthetic data	Artificially generated data used to exercise, demonstrate or test a system; usefulness depends on realism and relevance.
Story points / T-shirt sizing	Relative sizing approaches for discussing work complexity or effort; they must still be related to delivery capacity and timelines.
Traceability	The ability to connect requirements to design decisions, implementation, tests and evidence of completion.
CI/CD	Continuous Integration / Continuous Delivery (or Deployment): automated build, test and release pipelines that keep changes integrated and releasable.
MQTT broker	A message broker implementing the MQTT publish/subscribe protocol, commonly used for IoT device messaging.
Apache Kafka	A distributed event-streaming platform used for durable, high-throughput data pipelines and stream processing.
QFE (Quality First Engineering)	The discipline of designing quality into the process through explicit standards, structured checks, test coverage and release gates — rather than treating quality as an end-of-project activity or an individual developer's virtue.

15. Key Messages to Remember

1 Code generation is easier; engineering judgement is more valuable.

Define the right problem, structure the information, validate the solution and own the outcome.

2 Start with clarity.

Requirements, constraints, architecture, data and acceptance criteria should be understood before major implementation.

3 Treat data as a core product asset.

The right data enables useful insights, compelling demonstrations, reliable decisions and meaningful testing.

4 Define the schema before generating at scale.

A good structure makes outputs more consistent, understandable and testable.

5 Discover before defining.

Explore source material, ask questions and challenge assumptions rather than rushing into the first plausible design.

6 Use AI critically.

Multiple models can help challenge a proposal, but the final decision must be reasoned and verified.

7 Quality must be systemic.

Test cases, data dictionaries, traceability and release gates should protect the product regardless of who builds it.

8 Delivery includes time and accountability.

A technically correct solution delivered too late may still fail the customer's need.

9 Iterate until the process becomes repeatable.

A documented approach becomes more valuable when teams can apply it consistently.

10 Demonstrate outcomes, not just interfaces.

Use realistic data and a meaningful story to show why the application matters.

11 The schema is the higher-order design work

— it determines how the problem and its data will be organised before AI generates at scale.

12 Inspect actual data, not only field names or diagrams.

A schema is a hypothesis until its generated artefacts and tests have been validated.

13 Use peer review and multiple LLMs to discover omissions

while retaining responsibility for the final decision.

14 Quality is a rule; the number of review sources and the target of roughly three iterations are recommendations

to apply with judgement.

15 The goal is a defined, repeatable process first

then a mature and optimised process that can support higher-order automation.

Closing Note

The core message of the Mentor's sessions is a change in **professional identity**. In an AI-enabled development environment, engineers should not measure their contribution only by how much code they personally write. Their value lies in:


Understanding the domain


Defining precise structures


Making sound decisions


Building reliable data and tests


Guiding AI toward useful outputs


Taking accountability for delivery

The practical challenge for students is to **demonstrate these habits** in the accelerator project — not merely to describe them.

Developed by Talenciaglobal. Premium Talent Accelerator Program — Student Learning Notes. 16 October 2026.

16. Consolidated Session Notes: Schema-Driven Engineering, Process Maturity and QFE Execution

This session develops the earlier notes on schema-first and data-driven engineering into a practical method. The central argument: when AI can generate large volumes of code and documentation, the higher-value engineering work is to **discover the right structure, define it clearly, inspect the underlying data, validate the generated artefacts, and improve the process until it is repeatable**.

16.1 The four working principles emphasised

Discover before defining.

Study the source material, inspect examples and data, ask questions, and identify useful ways to organise the problem before committing to a schema.

Define nonlinear schemas.

Represent complex information as a graph or tree of connected concepts, categories, subcategories and attributes. Do not force the whole problem into a large flat list or relational table too early.

Use multiple sources of truth.

Challenge the first design with peers and more than one LLM/model. Three sources is a useful recommendation, not an inflexible rule.

Build a quality and review process.

Define what good looks like, test outputs against it, and move from an individual, improvised approach towards a defined and repeatable process.

16.2 Start by Inspecting the Generated Requirements-Analysis Docket

The Mentor revisited a requirements-analysis docket generated from a source requirements document, organised into approximately **27 sheets or major sections**, including scope, functional requirements, non-functional requirements, data domains, business rules, traceability, panel readiness, execution planning, decisions, issues and related areas. The numbering made the breadth of the analysis visible.

The functional-requirements section contained roughly **18 fields** — e.g., functional-requirement ID, functional domain, module, requirement, detailed description, actor, trigger, business rule, expected outcome, priority, coverage, tier, obligation, milestone, evidence class and status. Some fields were immediately understandable; others were not. **Do not guess what an unfamiliar label means.** Open the artefact, inspect actual values, filter and compare records, and ask what the field is intended to represent.

For example, the field "**evidence class**" appeared to include values such as *conflict*, *source-derived*, *source-explicit* and *unspecified*. Those labels require interpretation in context. A field name alone is not enough to establish its meaning or use. Likewise, fields such as "**obligation**" or "**status**" should not be accepted as self-explanatory without checking their definitions and sample values.

Practical lesson: Do not build a large application around hundreds or thousands of fields without first examining representative data. The schema and the data must be understood together.

16.3 Discover the Schema Hidden Inside the Document

A generated docket is not just a document: **it embodies a schema**. Its top level may contain around 27 sections; each section can contain its own fields; and some fields or categories may need to be divided into more specific subtypes (e.g., acceptance criteria may need their own classification and further subcategories).

The Mentor recommended turning this structure into a **graph**: the root represents the activity (e.g., requirements analysis); branches represent major sections; lower levels represent fields, subtypes and detailed attributes. A graph can have multiple roots or connected components — a **forest of graphs** is acceptable if that better represents the problem. The goal is sufficient and meaningful representation, not forcing everything into one linear sequence.

A flat prompt

Asks an LLM to read a source document and produce a requirements docket.

A schema-driven prompt

Goes further: it provides the source document *and* specifies the sections, fields, relationships and classifications expected in the output. The model then generates an artefact within a structure the team has deliberately designed.

Key distinction: Generating a document is comparatively easy with AI. **Discovering and defining the schema** that makes the document useful, complete, consistent and reviewable is the higher-order engineering task.

16.4 Why Nonlinear Schemas Matter

Complex domains do not always fit a neat linear list. A linear representation can become unwieldy, repetitive and difficult to navigate. A nonlinear schema lets engineers start with entities, categories, relationships and attributes, and refine the structure as understanding improves — analogous to thinking in terms of an **entity-relationship diagram** before deciding how to map into relational tables.

- Begin with the concepts and relationships that make sense in the domain.
- Separate major branches and subtrees so reviewers can understand the structure without being overwhelmed by leaf-level detail.
- Add lower-level attributes only when they contribute useful precision.
- Keep the structure open to refinement; the first schema is a hypothesis, not a final truth.
- Convert or map the schema into tables, code structures or other implementation forms when the conceptual model is sufficiently clear.

16.5 The Five Schemas Students Were Asked to Create

The Mentor asked teams to work on five related schema/graph artefacts. These should not be treated as five isolated deliverables — they should inform one another:

Schema	What it should organise
1. Requirements analysis	Scope, functional requirements, non-functional requirements, data domains, business rules, traceability, decisions, issues and related analysis areas.
2. Stakeholders and personas	Who is involved, their roles, needs, goals, constraints, responsibilities and relevant interactions.
3. Goals	The outcomes the system or project must achieve, how those outcomes relate, and what evidence would demonstrate success.
4. Feature discovery	Candidate features, their purpose, user/business value, dependencies, relationship to goals and possible priorities.
5. Test cases	Test categories, subcategories, the behaviours and requirements covered, and the evidence needed to establish that the product works.

Cross-links matter: features should connect to goals; requirements should connect to test cases; stakeholder needs should influence requirements and acceptance criteria. Teams can exchange ideas and reuse strong elements from one another's structures.

16.6 Lakshya's Test-Case Graph: A Practical Example

Lakshya shared a graph that classified test cases into categories and subcategories, with a maximum of about five child nodes at a level. The intention was to organise the test space first and then derive or connect the requirements the tests would cover. The Mentor welcomed the example and advised presenting the **top-level branches and subtrees first**, temporarily hiding leaf-level items — making it easier to review the organising logic before inspecting individual test cases.

The discussion explored possible test-graph branches. These were **suggestions for consideration, not a final mandated taxonomy**:

Structure and project best practices

Naming conventions, keys, domains and types, audit columns, physical and analytical objects, and consistency of the project/data structure. The Mentor related this to white-box-style testing.

Data integrity and data quality

Whether data is complete, valid, consistent, correctly related and fit for use. The Mentor suggested integrity and quality might deserve separate branches if their concerns are distinct.

Business rules and behaviour

Whether rules, expected behaviours and business outcomes are correctly enforced.

Security, privacy and access

Security controls, privacy requirements, permissions and access boundaries. These may be split into separate branches if the project requires that level of detail.

Non-functional requirements

Performance, scalability, latency, recovery, maintainability, accessibility, usability and other relevant quality attributes. The structure should reflect the actual system and its risks.

The Mentor did **not** claim the graph was already proven correct. It was a useful **working hypothesis**. To evaluate it, the reviewer must inspect the actual test cases and the data — not only the picture of the graph. The structure should then be refined, and the tests executed against adequate examples.

16.7 Discover, Define, Validate and Iterate

Do not jump straight from an initial idea to a finished schema. Experts often reveal omissions, alternative classifications and important edge cases. Discover first, define second, and validate the definition against evidence.

01

Discover

Read source documents, inspect generated artefacts and sample data, identify unknown terms, ask questions and look for patterns.

02

Draft the graph

Identify the root, major branches, subtrees and the attributes that matter.

03

Challenge the draft

Ask peers and multiple LLMs/models to find gaps, contradictions and alternative ways to organise the problem.

04

Define

Decide which structure is useful for the actual project, and explain why the categories and relationships belong there.

05

Generate

Provide the source context and the schema to the LLM so it can produce a structured artefact.

06

Validate

Compare the output with the source material, actual data, requirements and test cases. Check completeness and meaning, not just formatting.

07

Refine

Update the schema and generated output based on what validation reveals; repeat as needed.

08

Reuse

Once the approach works, apply it to another module or project and measure whether it becomes faster and more consistent.

The Mentor recommended aiming for around **three iterations** as a useful working habit. This is a recommendation, not a rigid rule: if the first version is genuinely sufficient, unnecessary iterations should not be added merely to meet a number. Conversely, complex work should not be declared complete just because a first draft looks convincing.

16.8 Three Sources of Truth: Models and Peers as Review Partners

The Mentor recommended challenging a schema with multiple sources, often **three**. Students could compare feedback from different LLMs such as ChatGPT, Claude and DeepSeek, as well as from peers or other teams. The point is **not** to accept a majority vote or assume any model is always correct — it is to surface blind spots and alternatives one source may miss.

- Ask reviewers to critique the **schema itself**, not only the output generated from it.
- Ask what is missing, ambiguous, duplicated, incorrectly classified or not supported by the source.
- Compare suggestions and decide which fit the project's context.
- **Record and resolve meaningful conflicts** rather than silently choosing one answer.
- Use peer reviews across teams so the wider group benefits from several teams' reasoning.

16.9 Process Maturity: From an Improvised Approach to Optimisation

The Mentor observed that the students' work showed promising thinking but still appeared **personalised**: individuals were figuring out their own methods as they worked. The next step is to make the method explicit, improve it through use and apply it consistently.

Stage	Meaning in the training
Undefined	Work is improvised; each person may use a different approach. Avoid this state except when an urgent situation genuinely requires immediate action.
Defined	The team has documented phases, steps, schemas and expected outputs. The Mentor referred to an approach containing about **eight phases and 46 steps**.
Repeatable	The defined method has been used successfully more than once, across additional modules or projects, and produces sufficiently consistent results.
Matured	The process has been improved through experience; roles, checks and working practices are well established and reliably applied.
Optimised	The team systematically reduces effort, cost and token usage, automates parts of the process and builds reusable frameworks or meta-frameworks.

A defined process is **not sacred or permanently correct**. It should be implemented, reviewed and revised. The Mentor suggested that repeated cycles — potentially around **six uses** in some cases — can help a process become well understood, but this was an illustrative observation rather than a universal rule. The important transition is from a documented hypothesis to a method that demonstrably works and can be repeated.

The Mentor also described an ambitious optimisation goal for a hackathon-scale application: use the process to assemble a substantial application while keeping LLM token consumption and cost under tight control, with a stated illustrative target of **not exceeding US$200**. This was presented as an **optimisation challenge, not a guaranteed cost estimate** for every project.

16.10 QFE — Quality First Engineering

The underlying principle: **quality must be assessed explicitly and must not be treated as optional.**

The Mentor described a way to create a demanding **external benchmark**. For example, a student could ask an LLM to describe the expected capabilities and accomplishments of a principal data architect at LinkedIn, then use that role profile as a **simulated standard** for reviewing their own work. The student would submit the artefacts, answer questions to demonstrate understanding, and ask for a **scorecard** evaluating quality, clarity, usability and other relevant criteria. The purpose is to make the standard visible and expose gaps a friendly or superficial review might miss.

Approximate score bands discussed (an illustrative coaching rubric, **not** a formal universal grading standard):

Indicative score	Interpretation discussed
7/10	A useful first green/light-green result; a respectable initial standard.
8/10	A stronger aspiration; described as green or dark green.
9–10/10	Very strong results, difficult to achieve on the first attempt.
6/10	Amber: improvement is needed.
4/10	Red: significant weaknesses.
3/10	Blood red: very poor result.

The central idea is to **define the quality bar in advance** and keep improving towards it. A red or amber result is not a reason to hide the work; it is a starting point for learning. The Mentor's non-negotiable was to pursue genuine quality and be willing to hear the truth about the work.

Especially important: ask reviewers to assess the **scorecard/schema and the criteria themselves**, not merely to rate the artefact vaguely. A good scorecard makes the definition of "done" clearer and quality more repeatable.

16.11 The Engineer's Role Is Moving Up the Value Chain

Students should not measure their contribution only by how much code they write. AI can generate large amounts of code and documentation, but someone must determine **what should be generated, how information should be structured, what evidence is relevant, what quality means and whether the result is correct**. That requires domain understanding, critical thinking and the patience to inspect the data.

Do not behave like a **courier** who forwards a generated document without opening the box or understanding what it contains. Understand which fields are derived, explicit, mandatory, optional or uncertain; clarify what each one means. Use AI actively, but retain human responsibility for context, structure, interpretation and acceptance. Move towards **higher-order automation** after the manual process is defined and repeatable: automate the workflow, build reusable frameworks and, where justified, build frameworks that automate framework creation.

16.12 Collaboration Is Part of the Method

The Mentor asked teams to share their schemas and review one another's work. Each team should both give and receive feedback — allowing the group to combine useful ideas and learn from several approaches rather than duplicating effort or treating one team's first draft as the standard.

- Present the **first two levels** of the graph before overwhelming reviewers with leaf-level details.
- Explain the reasoning behind the branches and classifications.
- Ask reviewers to identify omissions, unnecessary subdivisions, overlapping categories and unclear labels.
- Share useful patterns with other teams and incorporate improvements when they fit the context.
- Ensure the person who generated an artefact **can explain it and defend its structure**.

16.13 Expectations, Working Rhythm and Milestones

The Mentor asked students to use the time before the next session to make tangible progress on the five schemas. He said the group would meet again on **Wednesday at 9:30 a.m.**, and indicated that the programme did not impose attendance or participation compulsion at this level. Students were invited to participate sincerely if they found the work useful and to use their time elsewhere if they did not. The freedom was paired with a clear expectation: **if participating, take the work seriously and be willing to accept honest quality feedback.**

The stated process goal was to reach a **defined and repeatable** state by the end of **October 2026**, then take on more difficult problems and combine the methods in **November**. These were programme goals expressed in the session, not claims that the milestones had already been achieved.

Students were cautioned against rushing into implementation and release. Young engineers can be eager to start coding, ship a build and move on before the structure has been thought through. The Mentor asked them to work **patiently and critically**, examining each element carefully. The goal is not to delay for its own sake; it is to avoid the larger cost of generating large volumes of work that nobody can reliably validate.

16.14 Updated Student Checklist for This Session

- Create the **five project schemas**: requirements analysis, stakeholders/personas, goals, feature discovery and test cases.
- Represent complex areas as **nonlinear graphs/trees** with clear roots, branches and subtrees.
- Review the source document and **inspect actual generated data** before finalising field definitions.
- Write down the meaning of unfamiliar fields and clarify ambiguous values rather than guessing.
- Show the **top-level graph to peers first**; then inspect lower-level detail and actual records.
- Ask at least one or two peers and, where useful, multiple LLMs to challenge the schema. **Three sources is a recommendation, not a rigid rule.**
- Use a **clear schema plus context in prompts** when generating structured artefacts.
- Validate generated outputs against source requirements, actual data and test cases.
- Use approximately **three iterations** where helpful; stop earlier if the artefact is demonstrably sufficient, and continue if it is not.
- Build a test suite covering **structure, data integrity/quality, business behaviour, security/privacy/access** and relevant non-functional requirements.
- Create an explicit **quality scorecard** and define what "done" means before accepting an artefact.
- **Exchange reviews** with other teams and incorporate useful feedback.
- Aim to make the approach **defined and then repeatable**, before investing in optimisation and meta-framework automation.

16.15 Consolidated Takeaway

The practical method is: **understand the problem and its data → discover the structure → define a nonlinear schema → challenge it with independent sources and peers → generate with AI using the schema and context → validate the result against evidence and tests → iterate → and repeat the method until it becomes dependable.** Only then should the team focus on optimising and automating the process itself.

Developed by Talenciaglobal. Premium Talent Accelerator Program — Student Learning Notes. 16 October 2026.

End of notes.

